

CODE WALKTHROUGH · 16 PAGES

How each webpage works

This book shows only the code that makes a page save, log in, or update the screen. It is not a dump of every file. Pair it with MediCore-Staff-Guide.pdf for logins and simple words.

The same 3 layers on every staff page

Layer	Files	Job
1. HTML	patients.html, payment.html, ...	Boxes on screen: forms, table ids, buttons.
2. Page JS	js/patients.js, js/ward.js, ...	What happens on click. Calls MC.set to save.
3. Shared JS	js/data.js, auth.js, app.js	Login, left menu, talk to SQLite.
4. Server	app.py + medicore.db	Checks the session and writes the database.

Scripts at the bottom of a staff page (always this order)

example: payment.html (end of file)

```
<script src="js/data.js"></script>
<script src="js/auth.js"></script>
<script src="js/app.js"></script>
<script src="js/payment.js"></script>
```

data.js loads first so MC.get / MC.set exist. auth.js adds login. app.js draws the teal menu. Then the page file (payment.js) wires the buttons.

SHARED — saving to the database

js/data.js · MC.get and MC.set

Every list (doctors, patients, rooms, invoices...) lives in memory (MC._cache) and on disk (medicore.db). Pages never talk to SQLite themselves. They call two functions.

Why this matters: one save path for the whole hospital.

js/data.js — read a list

```
MC.get = function (key) {
  var name = storeName(key);           // "patients", "rooms", ...
  var rows = MC._cache[name];
  return Array.isArray(rows) ? rows : [];
};
```

js/data.js — save a list (screen + database)

```
MC.set = function (key, rows) {
  var name = storeName(key);
  MC._cache[name] = rows;              // 1. update the screen cache
  fetch("/api/store/" + name, {        // 2. send the full list to Flask
    method: "PUT",
    credentials: "include",           // session cookie = who is signed in
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(rows)
  });
  return rows;
};
```

Flask receives PUT /api/store/patients, checks you are logged in, then writes JSON into SQLite. Dashboard and Reports call MC.get on the same keys, so their numbers match.

js/data.js — new IDs (PT-4407, INV-77125, ...)

```
MC.nextId = function (rows, prefix, start) {
  var nums = rows.map(function (x) {
    return parseInt(String(x.id).replace(/\D/g, ""), 10);
  });
  var n = Math.max.apply(null, nums) + 1;
  return prefix + n; // e.g. prefix "PT-" → "PT-4407"
};
```

SHARED — login and the teal menu

js/auth.js + js/app.js

Passwords never sit in the HTML. The browser posts Staff ID + password to Flask. A cookie is stored. Later saves send that cookie automatically (credentials: include).

js/auth.js — sign in

```
MC.login = function (id, password) {
  return fetch("/api/login", {
    method: "POST",
    credentials: "include",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ id: id, password: password })
  }).then(function (r) { return r.json(); })
  .then(function (data) {
    if (!data.ok) return { ok: false, error: data.error };
    MC._me = data.user;           // { id, name, department }
    return { ok: true };
  });
};
```

js/auth.js — block the console if not signed in

```
MC.requireAuth = function () {
  if (MC._me) return MC._me;
  location.href = "login.html";    // kick back to login
  return null;
};
```

Staff HTML files set data-auth="staff". app.js sees that, calls requireAuth, then paints the left menu from MC.NAV (Home first, then Dashboard ... Email & SMS).

every staff HTML — page title used by the header

```
<body data-auth="staff" data-page="patients.html">
  <div id="pageTitle">Patient details</div>
  <div id="pageActions">
    <button id="openCreate">Register patient</button>
  </div>
  <aside id="sidebar"></aside>    <!-- app.js fills this -->
  <header id="topbar"></header>  <!-- title + Live + Sign out -->
  <div class="content">...forms and tables...</div>
</body>
```

[HOME](#) · [index.html](#)

Public site, or staff menu after login

One file, two skins. `data-auth="home"`. If there is no session, `#publicRoot` shows (photos, Staff login). If there is a session, `#staffRoot` shows — same sidebar as Payments.

index.html — two roots

```
<body data-auth="home" data-page="index.html">
  <div id="publicRoot"> <!-- logged out -->
    
    <a href="login.html">Staff login</a>
  </div id="modGrid"></div>
</div>
<div id="staffRoot"> <!-- logged in: teal console -->
  <aside id="sidebar"></aside>
  <div id="modGridStaff"></div>
</div>
</body>
```

js/app.js — pick the skin

```
if (auth === "home") {
  if (MC.session()) {
    publicRoot.style.display = "none";
    staffRoot.style.display = "block";
    MC.renderStaffChrome(); // same header as Payments
  } else {
    staffRoot.style.display = "none";
    publicRoot.style.display = "block";
  }
  MC.fillModuleGrid(modGrid); // cards → login or real page
}
```

`fillModuleGrid`: if logged out, every department card href is `login.html`. If logged in, href is `patients.html`, `ward-allotment.html`, and so on.

STAFF LOGIN · login.html

Two fields. No hints. Talks to Flask.

The page never lists DOC-1001 or the password. A Show/Hide button only changes input type. On submit, bindLoginForm calls MC.login, then goes to dashboard.html.

login.html — the only fields

```
<form id="staffLogin">
  <label>Staff ID</label>
  <input id="loginId" autocomplete="username">
  <label>Password</label>
  <input id="loginPw" type="password" autocomplete="current-password">
  <p id="loginErr" class="error"></p>
  <button type="submit">Continue</button>
</form>
```

js/app.js — submit

```
MC.bindLoginForm = function (formId) {
  form.addEventListener("submit", function (e) {
    e.preventDefault();
    MC.login(loginId.value, loginPw.value).then(function (res) {
      if (!res.ok) { loginErr.textContent = res.error; return; }
      location.href = "dashboard.html";
    });
  });
};
```

Flask (next pages) compares the password to a hash in the users table. Same error text for a bad ID or a bad password, so nobody can guess which one failed.

DASHBOARD · [dashboard.html](#) + [js/dashboard.js](#)

No typing — only counting

The dashboard does not save. It reads lists other pages already saved, then fills four tiles and three tables.

js/dashboard.js — tiles from live lists

```
MC.ready(function () {
  var appts = MC.get(MC.KEYS.appointments);
  var patients = MC.get(MC.KEYS.patients);
  var rooms = MC.get(MC.KEYS.rooms);
  var invoices = MC.get(MC.KEYS.invoices);

  var onDuty = MC.get(MC.KEYS.doctors)
    .filter(function (d) { return d.available; }).length;

  var occ = rooms.reduce(function (a, r) {
    return a + Number(r.occupied || 0);
  }, 0);
  // then write HTML into #kpis, #qBody, #revBars, #smsList
});
```

MC.ready waits until /api/bootstrap has filled the cache. That is why the numbers match Patients, Ward and Payments.

PATIENTS · [patients.html](#) + [js/patients.js](#)

Register and edit — then MC.set

Why: one form both creates PT-4407 and updates an existing row (hidden editId).

js/patients.js — save

```
form.addEventListener("submit", function (e) {
  e.preventDefault();
  var name = document.getElementById("cName").value.trim();
  if (!name) return; // show inline error
  var all = MC.get(MC.KEYS.patients);
  var eid = document.getElementById("editId").value;
  var row = {
    id: eid || MC.nextId(all, "PT-", 4401),
    name: name,
    age: Number(document.getElementById("cAge").value) || 0,
    phone: document.getElementById("cPhone").value.trim(),
    department: document.getElementById("cDept").value,
    doctorId: document.getElementById("cDoctor").value,
    status: document.getElementById("cStatus").value
  };
  if (eid) all = all.map(function (x) { return x.id === eid ? row : x; });
  else all.unshift(row);
  MC.set(MC.KEYS.patients, all); // → PUT /api/store/patients
});
```

Search just filters the array in memory (no extra API). Ward and Bill links are normal hrefs to the other HTML files.

DOCTORS · doctors.html + js/doctors.js

Add, edit, off duty, remove, optional login

js/doctors.js — add a doctor

```
var newId = MC.nextId(all, "DOC-", 1007);
var canLogin = document.getElementById("cLogin").checked;
all.push({
  id: newId, name: name, department: dept,
  phone: phone, available: true, login: canLogin
});
if (canLogin) {
  fetch("/api/staff-login", { // create hashed password on server
    method: "POST", credentials: "include",
    body: JSON.stringify({ id: newId })
  });
}
MC.set(MC.KEYS.doctors, all);
```

js/doctors.js — remove (cannot remove yourself)

```
if (del === me.id) return; // row marked "You"
if (!confirm("Remove " + doc.name + "?")) return;
if (doc.login) {
  fetch("/api/staff-login/" + del, { method: "DELETE", credentials: "include" });
}
MC.set(KEY, rows().filter(function (d) { return d.id !== del; }));
```

Off duty only flips d.available. Appointment booking later filters available: true. Flask refuses DELETE if it is your own ID, or if you would delete the last login.

APPOINTMENTS · appointments.html + js/appointments.js

Book a slot, then change status on the queue

js/appointments.js — confirm booking

```
form.addEventListener("submit", function (e) {
  e.preventDefault();
  var all = MC.get(MC.KEYS.appointments);
  all.unshift({
    id: MC.nextId(all, "AP-", 5512),
    patient: name, phone: phone,
    doctorId: document.getElementById("cDoctor").value,
    department: document.getElementById("cDept").value,
    date: document.getElementById("cDate").value,
    time: slot, // last time-chip clicked
    status: "Confirmed"
  });
  MC.set(MC.KEYS.appointments, all);
  MC.logAlert(phone, "Appointment Reminder", "Queued");
});
```

js/appointments.js — Confirm / Check in / Cancel

```
document.body.addEventListener("click", function (e) {
  var id = e.target.getAttribute("data-id");
  var st = e.target.getAttribute("data-st"); // "Checked-in"
  if (!id || !st) return;
  MC.set(KEY, rows().map(function (a) {
    if (a.id === id) a.status = st;
    return a;
  }));
});
```

Doctor dropdown is filled from MC.get(doctors) where available is true, and filtered by department.

PHARMACY · [pharmacy.html](#) + [js/pharmacy.js](#)

Stock goes up and down in the same list

js/pharmacy.js — low vs out

```
function statusOf(r) {  
  if (r.stock <= 0) return "Out";  
  if (r.stock <= r.min) return "Low"; // reorder level  
  return "In-stock";  
}
```

js/pharmacy.js — add / dispense / restock

```
// Add medicine  
all.unshift({  
  id: MC.nextId(all, "RX-", 10), name: name, batch: batch,  
  stock: Number(pStock.value) || 0, min: Number(pMin.value) || 10, unit: unit  
});  
MC.set(KEY, all);  
  
// Dispense 1 or Add 10 (buttons data-d / data-a)  
MC.set(KEY, rows().map(function (r) {  
  if (r.id === d) r.stock = Math.max(0, r.stock - 1);  
  if (r.id === a) r.stock += 10;  
  return r;  
})));
```

WARD ALLOTMENT · ward-allotment.html + js/ward.js

Add a room, allot a patient, never double-book

js/ward.js — add bed

```
all.push({
  id: id.toUpperCase(),           // e.g. PR-220
  type: "Private", floor: "2nd",
  beds: beds, occupied: 0, tariff: tariff,
  occupant: "", status: "Available", patients: []
});
save(all);                      // save() → derive() then MC.set(rooms)
```

js/ward.js — allot (block occupied + same patient twice)

```
var taken = rooms.some(function (r) {
  return (r.patients || []).some(function (p) {
    return p.toLowerCase() === patient.toLowerCase();
  });
});
if (taken) return toast("already has a bed");
if (freeBeds(room) <= 0) return toast("No free bed");
room.patients.push(patient);
save(rooms);
```

derive() sets Occupied / Partially Full / Available from patients.length vs beds. Discharge pops a name; if the room is empty, status becomes Cleaning.

DIAGNOSTICS · diagnostics.html + js/diagnostics.js

Book a scan, mark it done

js/diagnostics.js — book + complete

```
all.unshift({
  id: MC.nextId(all, "DX-", 2201),
  patient: patient,
  test: document.getElementById("cTest").value,    // MRI / CT / X-Ray ...
  slot: document.getElementById("cSlot").value,
  date: document.getElementById("cDate").value,
  status: "Scheduled"
});
MC.set(KEY, all);
MC.logAlert(patient, "Scan slot booked", "Queued");

// Mark done
MC.set(KEY, rows().map(function (r) {
  if (r.id === id) r.status = "Done";
  return r;
}));
MC.logAlert("", "Report Ready", "Queued");
```

PAYMENTS · [payment.html](#) + [js/payment.js](#)

Collect, create invoice, refund, receipt

js/payment.js — quick collect

```
var st = (qMethod === "Insurance") ? "Processing" : "Paid";
var byId = all.filter(function (x) { return x.id === ref; })[0];
if (byId && (byId.status === "Due" || byId.status === "Processing")) {
  byId.status = st; byId.method = qMethod; byId.amount = amt;
} else {
  all.unshift({
    id: MC.nextId(all, "INV-", 77120),
    patient: ref, amount: amt, method: qMethod,
    date: MC.today(), status: st, department: "General"
  });
}
MC.set(MC.KEYS.invoices, all);
MC.logAlert(ref, "Payment receipt", "Queued");
showReceipt(id); // print-friendly modal
```

KPI tiles re-sum the invoice list: Paid today, Due, Insurance, Refund. Dashboard revenue bars use the same invoices grouped by department.

REPORTS + EMAIL & SMS

js/reports.js · js/notifications.js

Reports never save. It only adds numbers from invoices, rooms and appointments. Email & SMS is a log. logAlert is used by booking, scans and payments too.

js/reports.js — occupancy and collections

```
var collected = invoices
  .filter(function (i) { return i.status === "Paid"; })
  .reduce(function (a, i) { return a + Number(i.amount); }, 0);
var beds = rooms.reduce(function (a, r) { return a + Number(r.beds); }, 0);
var occ = rooms.reduce(function (a, r) { return a + Number(r.occupied); }, 0);
var pct = beds ? Math.round(occ / beds * 100) : 0;
```

js/data.js — MC.logAlert (used by many pages)

```
MC.logAlert = function (to, template, status) {
  var rows = MC.get(MC.KEYS.alerts);
  rows.unshift({
    id: "AL-" + Date.now(),
    to: to, template: template,
    time: new Date().toString().slice(0, 5),
    date: MC.today(),
    status: status || "Queued"
  });
  MC.set(MC.KEYS.alerts, rows.slice(0, 80));
};
```

SERVER · app.py

Login check and SQLite write

This is the only place passwords are checked. They are stored hashed. The HTML never sees the hash.

app.py — login (same error if ID or password is wrong)

```
@app.post("/api/login")
def api_login():
    staff_id = request.json.get("id", "").strip().upper()
    password = request.json.get("password", "")
    row = db().execute(
        "SELECT id, password_hash FROM users WHERE id=?", (staff_id,)
    ).fetchone()
    if not row or not check_password_hash(row["password_hash"], password):
        return jsonify({"ok": False,
                        "error": "Invalid staff ID or password"}), 401
    session["uid"] = row["id"] # HttpOnly cookie
    return jsonify({"ok": True, "user": doctor_public(row["id"])})
```

app.py — save any module list

```
@app.put("/api/store/<name>")
@login_required
def api_put_store(name):
    body = request.get_json() # must be a JSON list
    save_store(name, body) # UPDATE stores SET payload=...
    return jsonify({"ok": True})
```

login_required returns 401 if the cookie is missing. Five failed logins lock that Staff ID for two minutes (_LOCK dictionary in memory).

ONE CLICK, END TO END

From a button to medicore.db

Example: Register patient on Patients.

Step	Where	What happens
1	patients.html	You fill the form and press Save.
2	js/patients.js	Builds a row {id, name, ...}. Puts it in the patients array.
3	js/data.js MC.set	Updates MC._cache and PUT /api/store/patients with credentials.
4	app.py	login_required reads the session cookie. If ok, save_store writes JSON.
5	medicore.db	Table stores, name='patients', payload = the full list.
6	dashboard.js	Next visit, MC.get(patients) counts Admitted for the tile.

Same path on every module

Page	Key	Save call
Doctors	doctors	MC.set(doctors) + /api/staff-login if they can sign in
Ward	rooms	save(rooms) after allot / add bed / discharge
Payments	invoices	MC.set(invoices) after collect / create / refund
Appointments	appointments	MC.set(appointments) + MC.logAlert
Pharmacy	pharmacy	MC.set(pharmacy) after stock change
Diagnostics	diagnostics	MC.set(diagnostics) + MC.logAlert
Email & SMS	alerts	MC.set(alerts) or MC.logAlert

Open the full files in Desktop/MediCore-Pages if you need a line the excerpt skipped. Staff logins and simple page words are in MediCore-Staff-Guide.pdf.